A Case Study

On

Air Quality Monitoring Network

Using IoT

Course Name: Internet Of Things

Course Code:CMP370

Program: B.E. Information Technology

Semester: Sixth Semester

Submitted By: Arjun Mandal

Roll No: 231506

Submitted To: Rishi K. Marseni

Date: 15 July 2026

Department of Information Technology

NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur, Nepal.

Table of Contents

List of Figures

1.Introduction

IoT connects sensors, actuators, and cloud services to enable continuous environmental monitoring. Urban air quality monitoring is critical for public health and policy; PM2.5, CO_2, and VOCs are key indicators linked to respiratory and cardiovascular risks. Traditional reference stations are accurate but sparse and costly; **low-cost sensor networks** provide dense spatial coverage and near-real-time data, enabling localized interventions and community awareness. This case study examines a city-scale **Air Quality Monitoring Network** built from ESP32 sensor nodes measuring PM2.5, CO_2, and VOCs, aggregated to a cloud dashboard. The focus is on system design, calibration and validation strategies, communication choices, and operational trade-offs—balancing cost, accuracy, and scalability to deliver actionable environmental intelligence.

2.Background Study

Urban monitoring gaps arise from the high cost and limited density of reference stations. Low-cost sensors (optical PM, NDIR CO_2, metal-oxide VOC) enable dense deployments but introduce sensor drift, cross-sensitivity, and environmental bias (temperature/humidity). Effective solutions combine co-location calibration with reference instruments, statistical or ML correction models, and robust data validation pipelines. Cloud platforms (AWS, Firebase, InfluxDB/TimescaleDB) support ingestion, processing, and visualization; Chart.js and Leaflet enable interactive dashboards and heatmaps. Case studies show hybrid networks (dense low-cost nodes + sparse references) improve exposure mapping and public advisories, but require ongoing maintenance, recalibration, and governance. This motivates a modular architecture with automated calibration, stream processing, and clear operational procedures.

3.Problem Statement

Cities lack sufficient granular, timely air quality data to identify local hotspots and support targeted mitigation. Reference stations are accurate but too few; raw low-cost sensors are noisy and drift over time. The objective is to design a scalable, low-cost monitoring network that provides validated PM2.5, CO_2, and VOC measurements for city-scale mapping and decision support. The system must enable automated calibration, data validation, and accessible visualization so municipalities, researchers, and citizens can rely on the data for health advisories and planning.

4.System Architecture

The architecture uses a three-layer model: Device Layer (ESP32 nodes + sensors), Network Layer (Wi-Fi / MQTT / HTTP), and Cloud Layer (ingest, calibration, storage, dashboard). Each ESP32 samples sensors, applies local preprocessing (median filter, temp/humidity compensation), timestamps readings (NTP), and publishes JSON telemetry via MQTT. The MQTT broker forwards messages to stream processors that validate payloads, apply calibration models (linear or ML), and persist cleaned records to a time-series database (InfluxDB or TimescaleDB). A REST API (FastAPI/Flask) exposes historical queries, device management, and firmware distribution; the frontend (Chart.js + Leaflet) renders heatmaps, time series, and node health. Security is enforced with TLS, device authentication (certificates or keys), and RBAC for the dashboard. The design supports modular upgrades: LoRaWAN or cellular gateways for connectivity gaps, edge ML for local anomaly detection, and automated OTA updates. The included architecture diagram shows node placement, network flow, and cloud components.

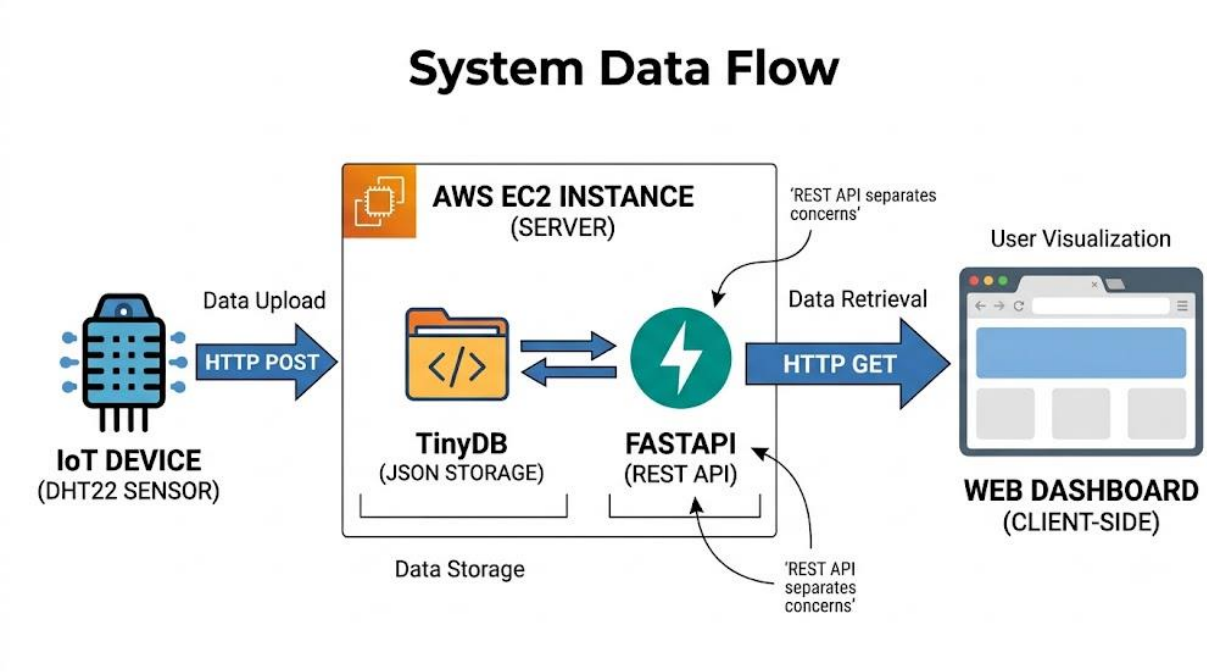

Fig 4.1. System Data Flow

5.Hardware Components

ESP32: chosen for integrated Wi-Fi, sufficient GPIO, low cost, and deep-sleep power modes. PM2.5 sensor (optical) provides µg/m³ estimates but needs humidity compensation and periodic cleaning. CO_2 sensor (NDIR) offers selective ppm measurements with better stability but higher cost. VOC sensor (metal-oxide or IAQ module) gives an index or ppb estimates and is cross-sensitive. Power options include mains, solar + battery with charge controller for remote nodes. Enclosures should allow airflow while protecting electronics. Optional actuators (fans, relays) can support active sampling or demonstration mitigation. A concise sensor-connection diagram maps UART/I²C/SPI lines, 3.3V/GND, and status indicators.

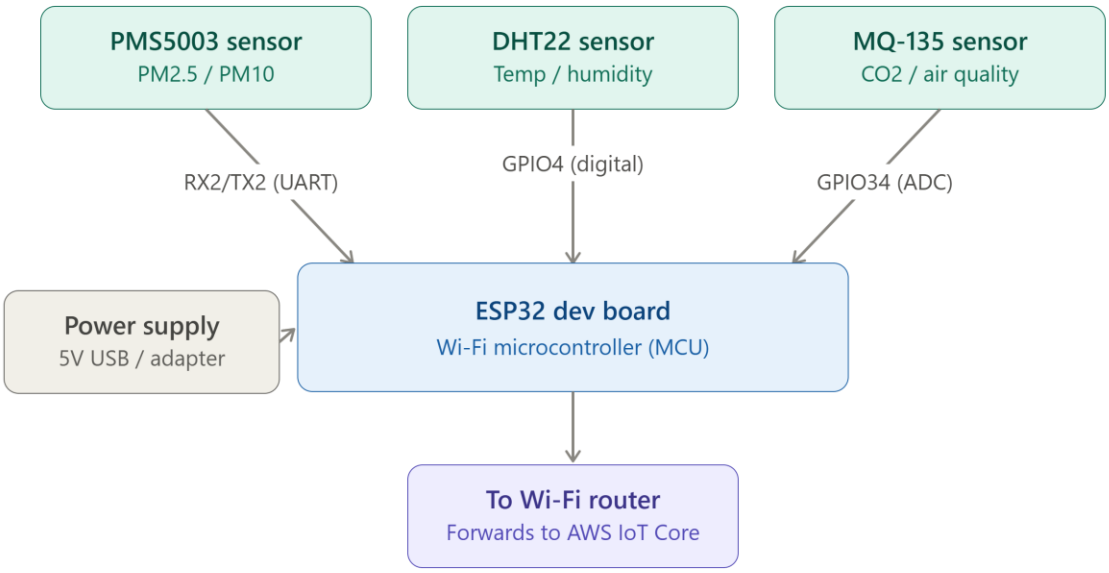

Fig 5.1 ESP32 Connection

6.Software Components

Device firmware (Arduino/C++ or ESP-IDF) handles sampling, local filtering, NTP sync, and secure telemetry (MQTT/HTTPS) with OTA support. Backend uses FastAPI/Flask for device management and REST endpoints; an MQTT broker (AWS IoT Core or Mosquitto) ingests telemetry. Stream processing (Lambda/Kinesis or containerized consumers) validates and calibrates data; calibration models use scikit-learn/XGBoost. Storage: InfluxDB or PostgreSQL+TimescaleDB for time-series; PostgreSQL/MySQL for metadata. Dashboard: HTML/JS with Chart.js for charts and Leaflet for maps. DevOps: containerization (Docker), CI/CD pipelines, and monitoring (Prometheus/Grafana). Security: TLS, API keys/OAuth, RBAC, and signed OTA updates.

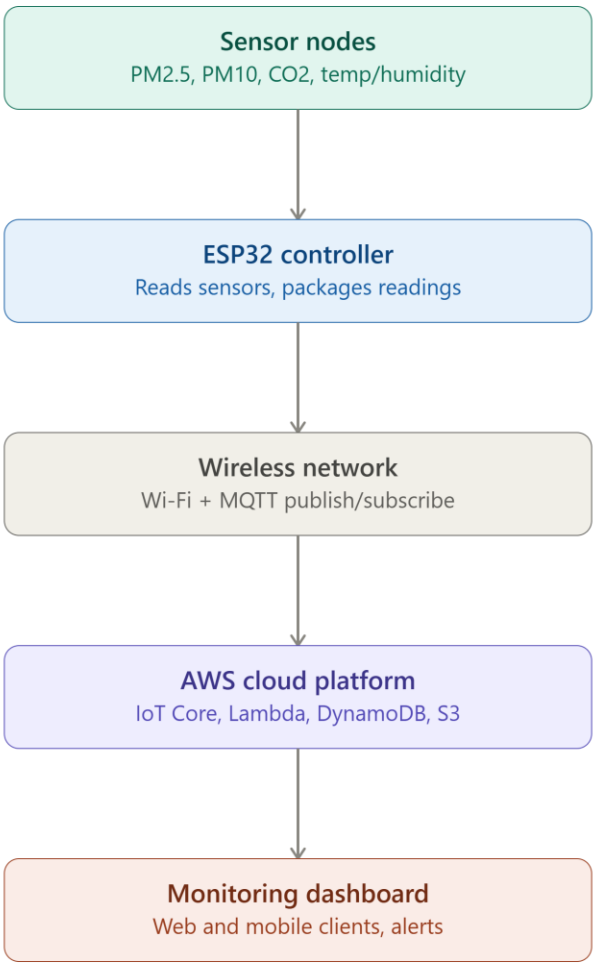

Fig 6.1 IoT workflow

7.Communication protocol

MQTT is preferred for telemetry due to low overhead, publish/subscribe scalability, and QoS options; use MQTTS for encryption. HTTP/REST suits device provisioning, firmware downloads, and bulk uploads. WebSocket supports real-time dashboard updates. CoAP is an option for constrained UDP networks but has less ecosystem support. For city-scale deployments, a hybrid approach—MQTT for frequent telemetry and REST for management—optimizes bandwidth, power, and reliability. Broker clustering and load balancing ensure scalability.

8.Dataflow Analysis

Sensors → ESP32 (sampling, local filtering, timestamping) → MQTT broker → stream processors (schema validation, range checks, calibration) → time-series DB + cold archive (S3) → REST API / WebSocket → Dashboard (heatmaps, time series) → Users/Alerts. Validation steps: schema checks, range and temporal consistency checks, cross-node correlation, and anomaly detection. Calibration workflows include periodic co-location with reference stations and automated recalibration jobs. Alerts trigger when thresholds are exceeded; exports support research and regulatory reporting. The Level-0 data flow diagram visualizes these interactions.

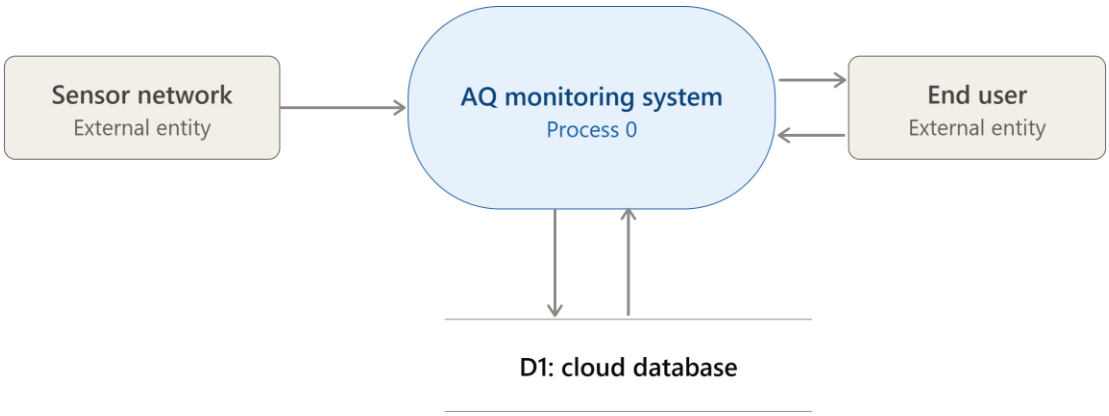

Fig 8.1 Dataflow Diagram

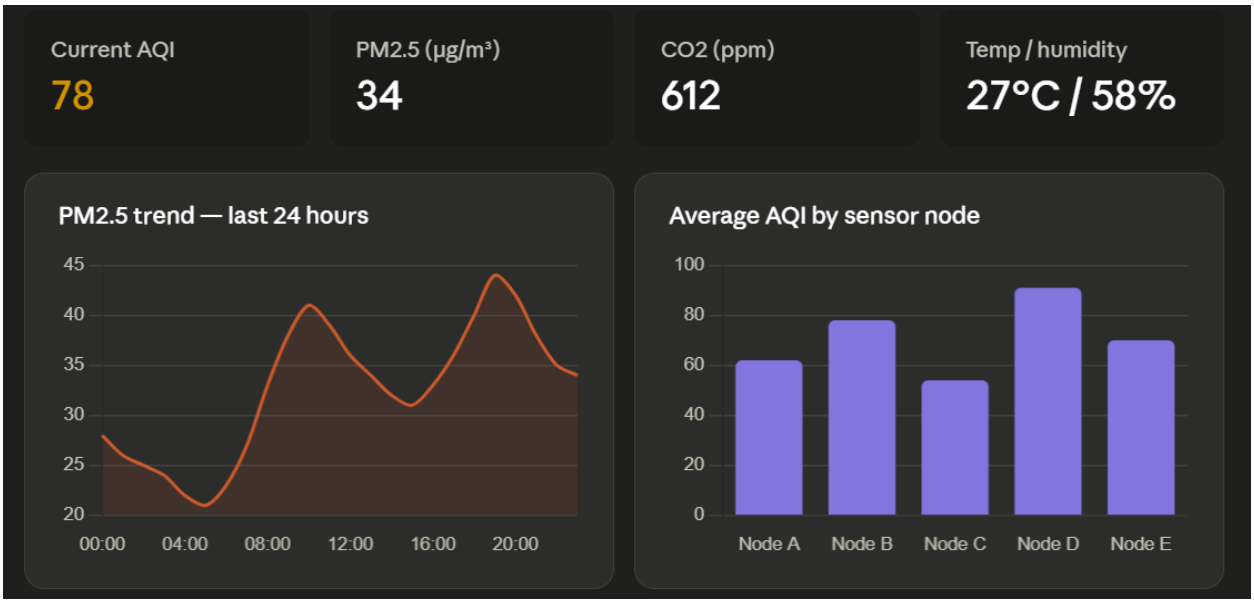

Fig 8.2 Dashboard

9.Security analysis

Threats: unauthorized access, data tampering, replay attacks, physical tampering, and credential compromise. Controls: TLS (HTTPS/MQTTS), device authentication (X.509 or unique keys), RBAC, API rate limiting, signed OTA updates, and secure boot where supported. Data integrity: message signing or sequence numbers, audit logs, and cryptographic hashes for critical records. Network: segmentation, firewalls, VPN for admin access, and anomaly detection. Operational: patching, incident response, backups, and privacy measures (aggregation/anonymization) for public dashboards. Regular security testing and compliance checks are required.

10. Mapping with Course Labs

Lab 1 (AWS EC2): Deploy dashboard and backend on EC2; configure security groups and monitoring. Lab 2 (REST API): Implement device registration, configuration, and data ingestion endpoints. Lab 3 (Dashboard): Build Chart.js time-series and Leaflet heatmaps; implement WebSocket or polling for real-time updates. Lab 4 (Embedded Devices): Program ESP32, interface sensors, implement NTP sync and MQTT publishing.

11.Challenges and Limitations

Sensor accuracy and drift require regular recalibration and cleaning. Environmental cross-sensitivity complicates interpretation. Connectivity and power limit placement; solar adds cost and maintenance. Operational costs (cloud storage, processing) scale with deployment size. Maintenance logistics for firmware updates, sensor replacement, and data governance are nontrivial. Public trust depends on transparent calibration and validation; false alarms can erode confidence.

12. Future Improvements

Implement ML-based calibration and anomaly detection, deploy edge computing for local validation and bandwidth reduction, and adopt hybrid connectivity (Wi-Fi + LoRaWAN/cellular) for resilience. Use digital twins and predictive analytics for hotspot forecasting. Consider blockchain for tamper-evident records in regulatory contexts. Expand sensor suite (meteorological sensors) and engage communities through open APIs and citizen science programs.

13. Personal Reflection

This project strengthened skills across embedded systems, cloud services, and data analytics. Hands-on work with ESP32 and sensors clarified hardware trade-offs; backend and dashboard development reinforced API and visualization skills. Calibration work highlighted the gap between raw sensor output and actionable data, motivating interest in ML and edge computing. The labs provided a structured path from device to cloud, making the end-to-end system tangible and educational.

14. Conclusion

A hybrid approach—dense low-cost ESP32 networks calibrated against reference stations and supported by robust cloud pipelines—delivers actionable, city-scale air quality insights. Key success factors are automated calibration, rigorous data validation, secure communications, and operational planning for maintenance and cost control. With targeted improvements (edge ML, hybrid connectivity), such networks can meaningfully augment public health monitoring and urban planning.

15. References

[1] J. Snyder et al., “Low-cost sensor networks for air quality monitoring: A review,” *Environ. Monit. J.*, 2020.

[2] A. Kumar and B. Singh, “Calibration methods for low-cost PM sensors,” *Sensors Actuators B*, 2021.

[3] M. Smith et al., “Hybrid networks: reference stations and low-cost sensors,” *Atmos. Meas. Tech.*, 2021.

[4] Plantower PMS5003 Datasheet, 2019.

[5] Senseair NDIR CO_2 Datasheet, 2020.

[6] D. Johnson, “MQTT for IoT,” *IoT Systems Mag.*, 2019.

[7] T. Brown et al., “Edge computing for environmental sensing,” *IEEE IoT J.*, 2020.

[8] InfluxData, “Time series data with InfluxDB,” Whitepaper, 2020.

[9] AWS IoT Core Documentation, Amazon Web Services, 2022.

[10] R. Patel and S. Lee, “ML approaches for sensor calibration,” *J. Environ. Inform.*, 2022.